

Estrategias de Chunking

Fragmentación inteligente de documentos para sistemas RAG con bases de datos vectoriales

Fixed-size

Sentence-aware

Semantic

Hoja de Ruta de la Clase

01 ¿Qué es Chunking y por qué importa?

El cuello de botella silencioso de los sistemas RAG

03 Fixed-size Chunking

Algoritmo, parámetros, ventajas y limitaciones

05 Semantic Chunking

Embeddings + detección de cambios temáticos

07 Integración con pgvector

Pipeline completo SQL + vectores en PostgreSQL

02 Anatomía de un Chunk

Tokens, overlap, metadata, fronteras semánticas

04 Sentence-aware Chunking

Respeto por límites oracionales y cohesión textual

06 Comparativa & Métricas

Cómo evaluar qué estrategia funciona mejor

08 Taller Práctico

Diseño, implementación y debate de trade-offs

¿Qué es Chunking y por qué es crítico en RAG?

El Problema

- Los LLMs tienen ventanas de contexto limitadas (4K–128K tokens)
- Un embedding por documento entero pierde especificidad
- Recuperar páginas completas introduce ruido irrelevante
- La calidad del chunk determina la calidad de la respuesta final

DOCUMENTO LARGO

CHUNKING

Chunk 1

Chunk 2

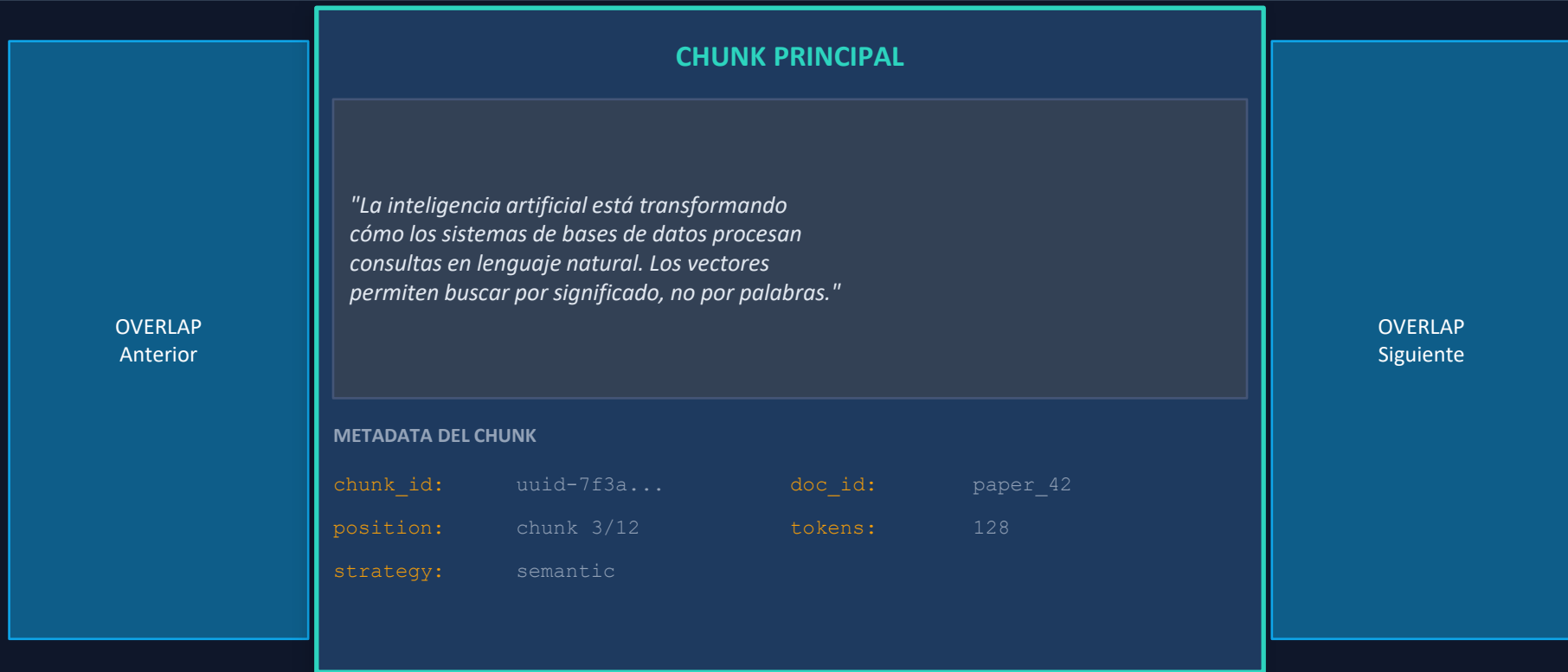
Chunk 3

El Objetivo

- Chunks lo suficientemente pequeños para ser específicos
- Chunks lo suficientemente grandes para tener contexto
- Fronteras que respeten la coherencia semántica
- Metadata que permita reconstruir el contexto original

Regla de Chunking: el chunk ideal contiene exactamente la información que necesita una consulta, ni más ni menos.

Anatomía de un Chunk



Overlap evita pérdida de contexto en fronteras

El chunk almacena texto + embedding + metadata

Estrategia 1 — Fixed-size Chunking

Visualización — *chunk_size=100 tokens, overlap=20 tokens*

Chunk A [t:0–100]

overlap
20t

Chunk B [t:80–180]

overlap
20t

Chunk C [t:160–260]

```
# Python — Fixed-size Chunking

from langchain.text_splitter import
    RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size    = 512,  # tokens
    chunk_overlap = 64,   # tokens
    length_function = len,
    separators = ['\n\n', '\n', ' ', '']
)

chunks = splitter.split_text(document)

# INSERT into pgvector
for i, chunk in enumerate(chunks):
    emb = embed_model.encode(chunk)
    db.execute(INSERT_SQL, (chunk, emb, i))
```

Parámetros clave

chunk_size: 256–1024 tokens (depende del modelo)

overlap: 10–20% del chunk_size

separators: jerarquía de corte (`\n\n > \n > ' '`)

Cuándo usarlo

Documentos homogéneos sin estructura clara

Logs, datos técnicos, textos repetitivos

Cuando la velocidad de indexación es prioritaria

Limitaciones

Puede cortar oraciones a la mitad

Chunks sin coherencia semántica interna

Overlap no garantiza continuidad de ideas

Complejidad: $O(n)$ · Velocidad: muy alta · Calidad semántica: baja-media

Estrategia 2 — Sentence-aware Chunking

Las oraciones se detectan primero; luego se agrupan sin romper ninguna

S1: Los transformers revolucionaron el NLP.

S3: El modelo BERT usa encoders bidireccionales.

S5: Ambos usan embeddings posicionales.

S2: Permiten atención sobre toda la secuencia.

S4: GPT usa decoders autorregresivos.

S6: La vectorización captura el significado.

Chunk 1 (2 oraciones)

Chunk 2 (2 oraciones)

Chunk 3 (2 oraciones)

```
# NLTK + agrupación por oraciones
import nltk
nltk.download('punkt')

def sentence_chunker(text, max_sent=5,
                    overlap_sent=1):
    sents = nltk.sent_tokenize(text)
    chunks = []
    i = 0
    while i < len(sents):
        chunk = sents[i:i+max_sent]
        chunks.append(' '.join(chunk))
        i += max_sent - overlap_sent
    return chunks
```

Tokenizadores disponibles

NLTK punkt: robusto para español/inglés
spaCy sentencizer: más rápido, pipeline completo
stanza: soporte multilinguaje avanzado
regex rules: para formatos muy controlados

Ventajas vs Fixed-size

Nunca corta a mitad de oración
Chunks más legibles y coherentes
Mejor recall en preguntas sobre hechos concretos

Complejidad: $O(n)$ · Velocidad: alta · Calidad semántica: media-alta

Estrategia 3 — Semantic Chunking

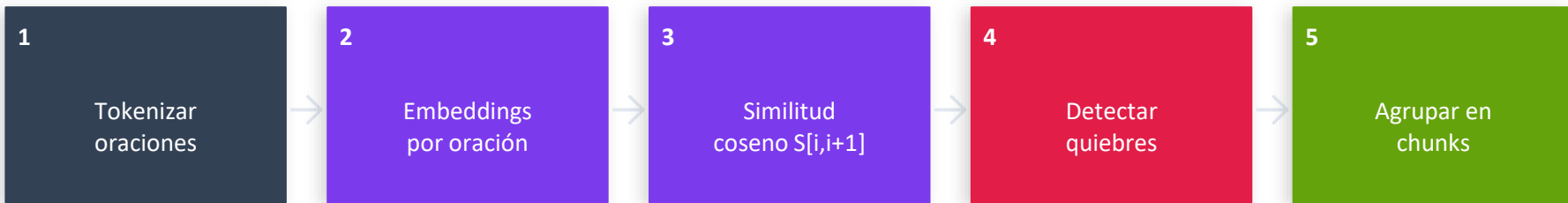
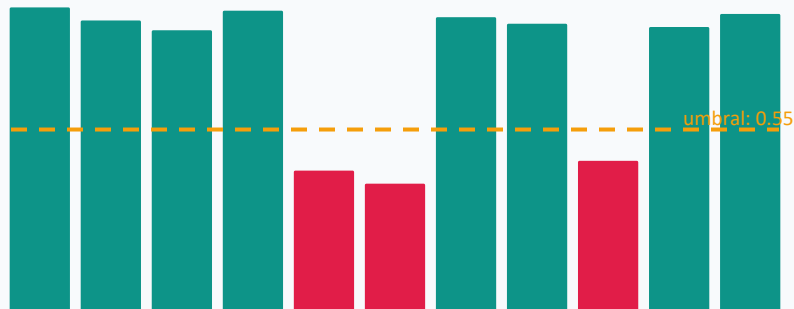


Gráfico de Similitud entre Oraciones Adyacentes



quebres semánticos (rojo = nuevo chunk)

```
# Semantic Chunking con sentence-transformers

from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')

def semantic_chunk(text, threshold=0.55):
    sents = sent_tokenize(text)
    embs = model.encode(sents)
    sims = cosine_similarity(embs[:-1],
                             embs[1:])
    breaks = [i+1 for i,s in enumerate(sims)
               if s[0] < threshold]
```

Complejidad: $O(n^2)$ · Velocidad: baja-media · Calidad semántica: muy alta

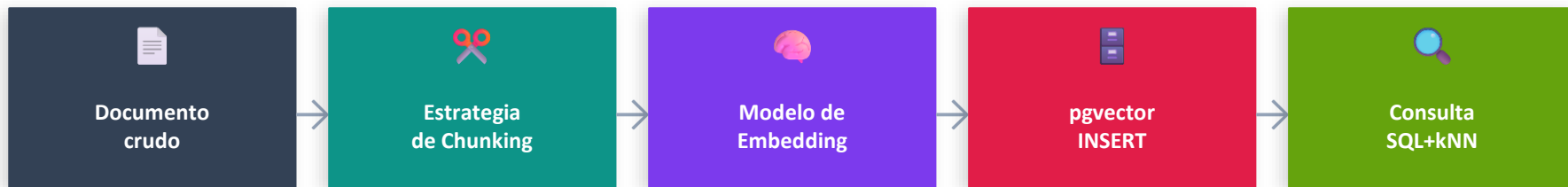
Comparativa de Estrategias & Métricas de Evaluación

Criterio	Fixed-size	Sentence-aware	Semantic
Velocidad de indexación	★★★★★	★★★★☆	★★☆☆☆
Coherencia semántica	★★★☆☆	★★★★☆	★★★★★
Facilidad de impl.	★★★★★	★★★★☆	★★★☆☆
Precisión en retrieval	★★★★☆	★★★★☆	★★★★★
Consumo de RAM/CPU	Bajo	Bajo	Alto
Soporte multilinguaje	Sí	Req. lib	Req. modelo
Recomendado para	Logs, datos homogéneos	Artículos, noticias	Papers, docs técnicos

¿Cómo medir la calidad del chunking?

Faithfulness · Answer Relevancy · Context Recall (framework: RAGAS)

Pipeline Completo: Chunking + pgvector en PostgreSQL



```
-- Schema PostgreSQL + pgvector

CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE documents (
  id          SERIAL PRIMARY KEY,
  doc_id      TEXT NOT NULL,
  chunk_index INT NOT NULL,
  strategy    TEXT, -- 'fixed'|'sent'|'sem'
  content     TEXT NOT NULL,
  metadata    JSONB,
  embedding   vector(384), -- MiniLM-L6
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- Índice HNSW para kNN semántico
CREATE INDEX ON documents USING hnsw
(embedding vector_cosine_ops);
```

```
-- Consulta híbrida: SQL + kNN vectorial
```

```
SELECT
  d.content,
  d.metadata->>'source' AS fuente,
  1 - (d.embedding <=> $1::vector)
    AS similitud
FROM documents d
WHERE
  d.strategy = 'semantic' AND
  d.metadata->>'category' = 'tech'
ORDER BY
  d.embedding <=> $1::vector -- coseno
LIMIT 5;
```

```
-- $1 = embedding de la consulta
-- <=> = operador coseno de pgvector
```

TALLER PRÁCTICO — Diseño & Comparación de Estrategias

Escenario: construir el pipeline de chunking para el sistema RAG del proyecto final

Tienen una colección de documentos mezclados: artículos académicos en PDF, noticias cortas, manuales técnicos y FAQs de producto. Cada grupo implementará y comparará las tres estrategias sobre el mismo corpus.

A

Análisis del Corpus

Reciben 4 documentos (paper, noticia, manual, FAQ). Para cada uno deben argumentar: ¿qué estrategia es más adecuada y por qué? ¿Qué chunk_size recomiendan? ¿Overlap o sin overlap?

B

Implementación en Python

Implementar las 3 estrategias sobre el mismo texto y mostrar el resultado. Contar cuántos chunks genera cada una, tamaño promedio y si algún chunk 'rompe' una idea importante.

C

Inserción en pgvector

Escribir el SQL completo: CREATE TABLE con la columna vector, el INSERT de los chunks con sus embeddings y la consulta kNN que devuelve los 5 chunks más relevantes para una pregunta dada.

D

Presentación & Debate

Cada grupo presenta su hallazgo más sorprendente. Debate: ¿chunk_size de 128 vs 512 tokens: qué gana y qué pierde? ¿Cuándo vale la pena el costo de semantic chunking?

Taller — Ejercicios de Implementación y Análisis

EJ 1 Fragmentar y medir

Fixed-size

Aplica fixed-size chunking con `chunk_size=256` y `overlap=32`. Luego con `chunk_size=512` y `overlap=64`. Compara: ¿cuántos chunks obtienes? ¿Cuál produce mejor recall al buscar un dato puntual del texto?

EJ 3 Ajustar el umbral semántico

Semantic

Con semantic chunking, prueba `threshold=0.4`, `0.55` y `0.7`. Muestra los chunks resultantes en cada caso. ¿Qué umbral produce chunks más útiles para el paper de ejemplo? Justifica con la curva de similitud.

EJ 5 Preguntas de análisis crítico

Análisis

¿Por qué un chunk de 20 tokens puede ser peor que uno de 0 tokens? ¿Qué pasa si dos chunks consecutivos son casi idénticos? ¿Cómo afecta el idioma del texto a la elección de estrategia?

EJ 2 Detectar fronteras oracionales

Sentence-aware

Implementa sentence-aware con NLTK y con spaCy. Compara los límites que detecta cada uno. Identifica al menos 1 caso donde NLTK falla y spaCy acierta (o viceversa).

EJ 4 Pipeline en PostgreSQL

pgvector

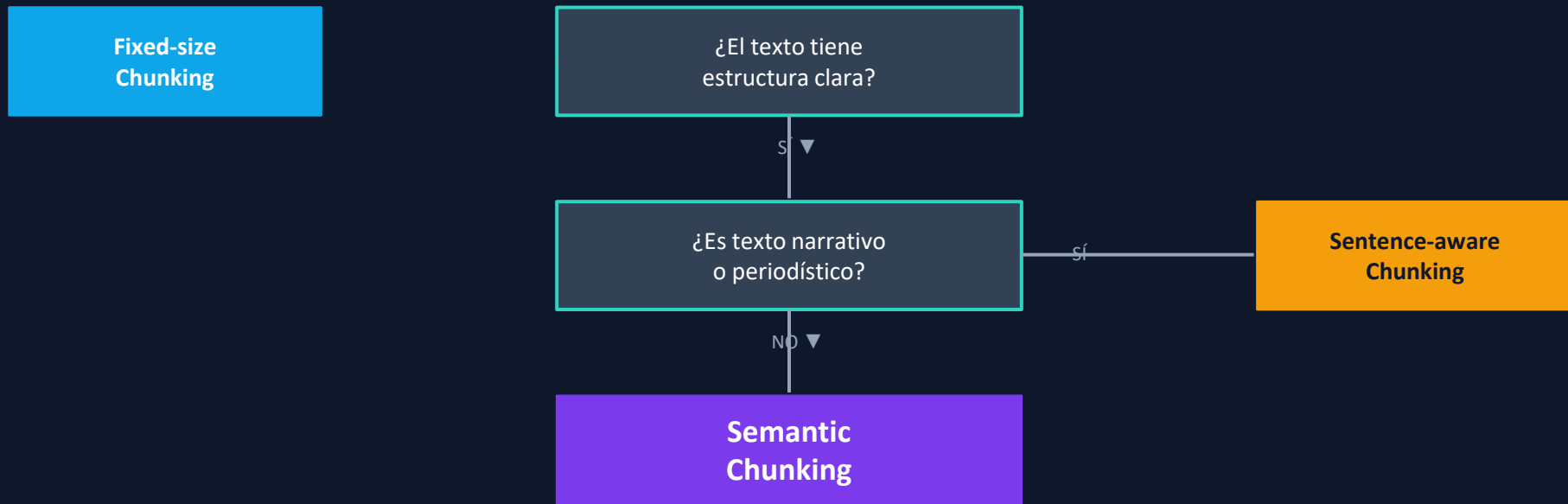
Escribe el CREATE TABLE, el script de inserción (Python + psycopg2 + pgvector) y la query híbrida que filtra por `doc_id` y ordena por similitud coseno. Test con la pregunta: '¿Qué es la atención multi-head?'

EJ 6 Bonus — Estrategia híbrida

Bonus

Diseña una estrategia que combine sentence-aware como base y luego aplica semantic merging para unir oraciones con similitud > 0.8 . ¿Mejora la precisión en retrieval? ¿A qué costo?

Resumen: Guía de Decisión para Chunking en RAG



El chunking es la decisión más impactante de un pipeline RAG — más que el LLM elegido.

Siempre evalúa con RAGAS o métricas propias antes de ir a producción.

Las estrategias se pueden combinar: sentence-aware como base + semantic merge.

pgvector con HNSW + SQL híbrido es el stack más pragmático para el proyecto final.